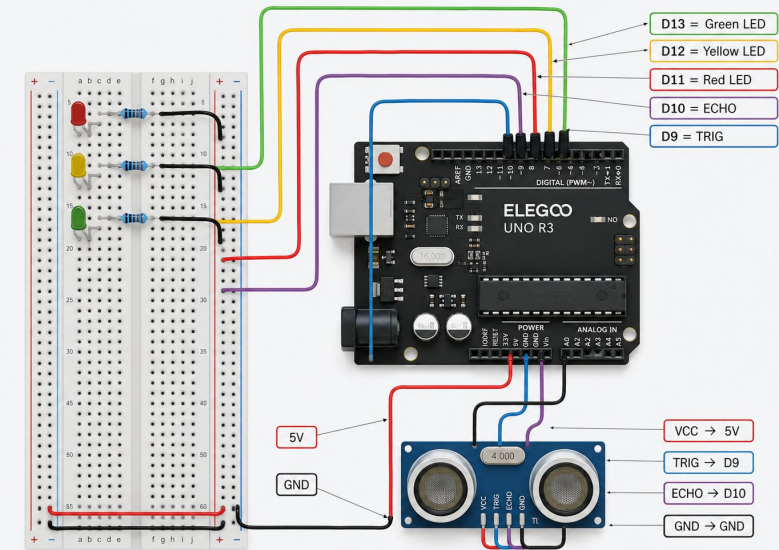


Traffic Light Optimization Simulator

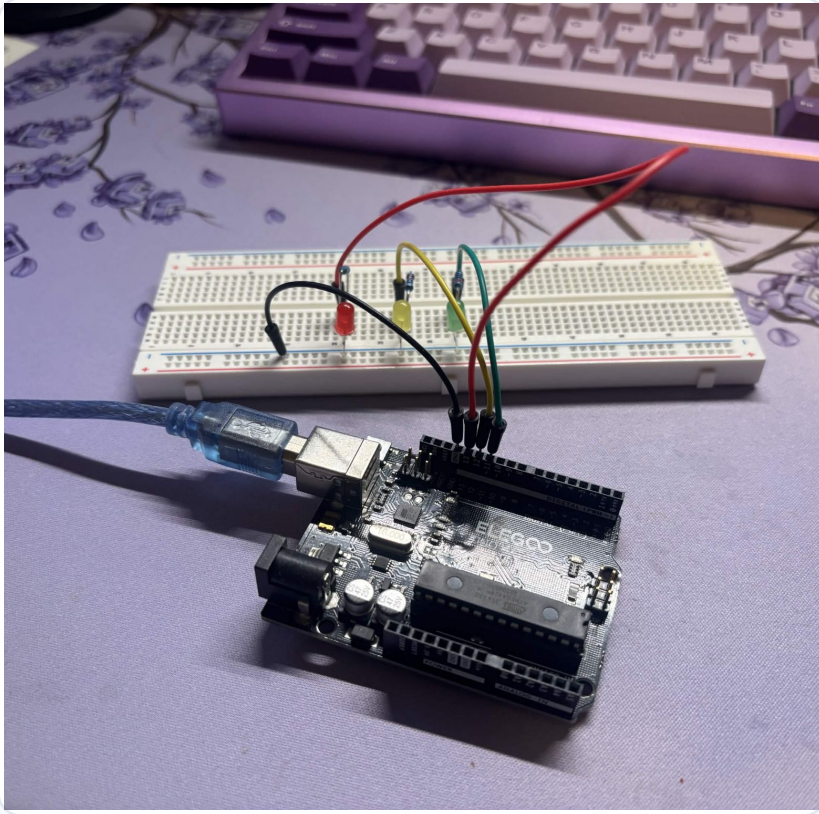
Final Presentation · EGG 104-1003

Zihan Chen · Calvin Le Zheng
Evan Macke · Nathaniel Lubas
University of Nevada, Las Vegas



Problem: fixed timing wastes time

Our goal was to make the traffic light respond to simulated congestion.



What we wanted to solve

Standard traffic lights use fixed timing even when traffic demand changes. This can create unnecessary waiting and make intersections less efficient.

Project goal

Build a physical simulator that counts Hot Wheels cars with an ultrasonic sensor, then changes the green, yellow, and red light timing based on the traffic level.

3

traffic levels

10 sec

counting window

Arduino IDE

best final control

Plan of Action vs. Actual Outcome

We started with a broad idea, then narrowed it into a working sensor-based prototype.

Original Plan

- Research traffic light logic and congestion conditions
- Use MATLAB/Simulink with Arduino and LEDs
- Create a simulator that responds to at least two traffic conditions
- Test, revise, and prepare a final demonstration



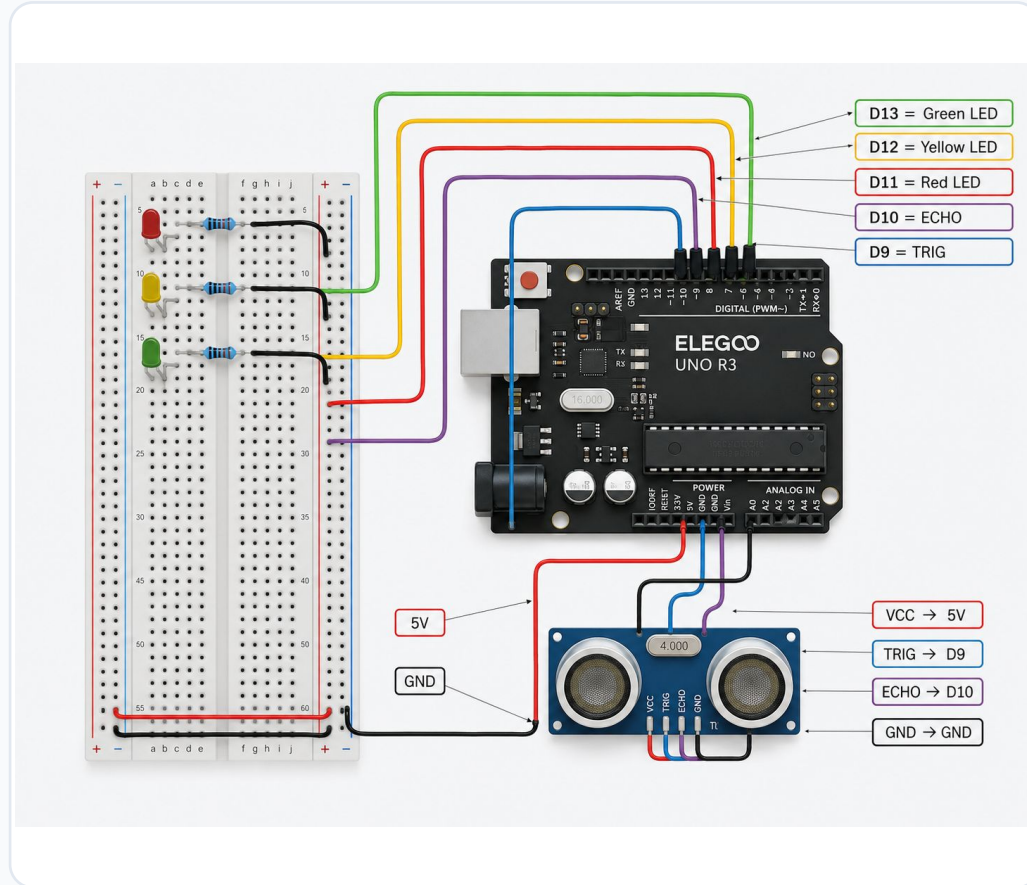
Actual Outcome

- Built a fixed-timing LED prototype first
- Added HC-SR04 ultrasonic sensing and Hot Wheels car counting
- Tested 0 cars, 1–2 cars, and 3+ cars with different timing
- Used Arduino IDE for final tests because it counted cars more reliably

Main change: the final prototype focused less on Simulink and more on a dependable Arduino IDE demonstration with real-time sensor input.

System Design: how the prototype works

The prototype uses one sensor input and three LED outputs.



1

Cars pass sensor

HC-SR04 measures distance.

2

Arduino counts cars

Each valid detection increments the count.

3

Timing is selected

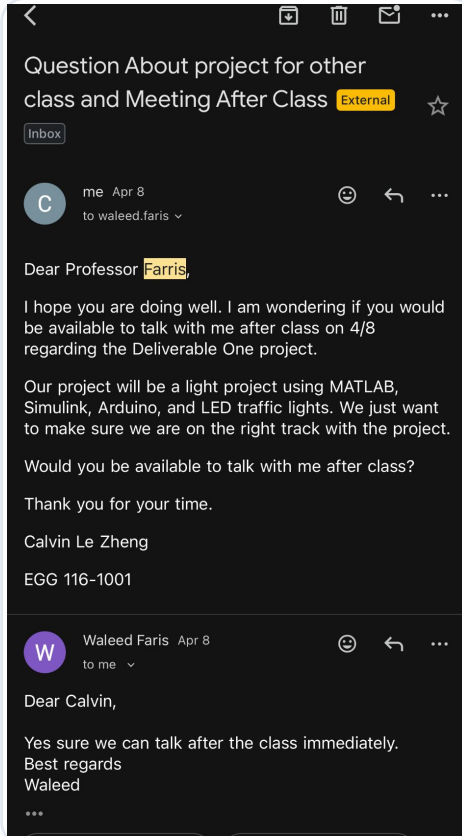
0, 1–2, or 3+ cars choose the cycle.

4

LEDs respond

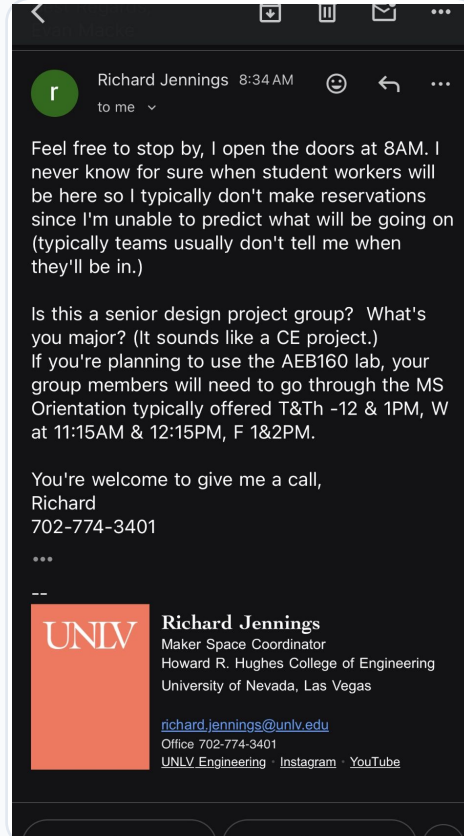
Green, yellow, and red use the selected durations.

Faculty and MakerSpace guidance shaped the build



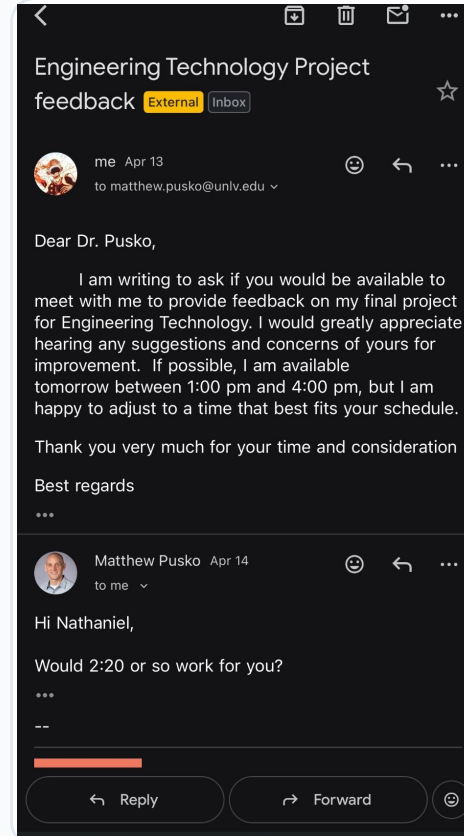
Dr. Faris

Make sensing and sensor placement a major design factor.



Richard Jennings

Keep the first build realistic, easy to test, and manageable.



Dr. Pusko

Focus on scalable logic and traffic detection methods.

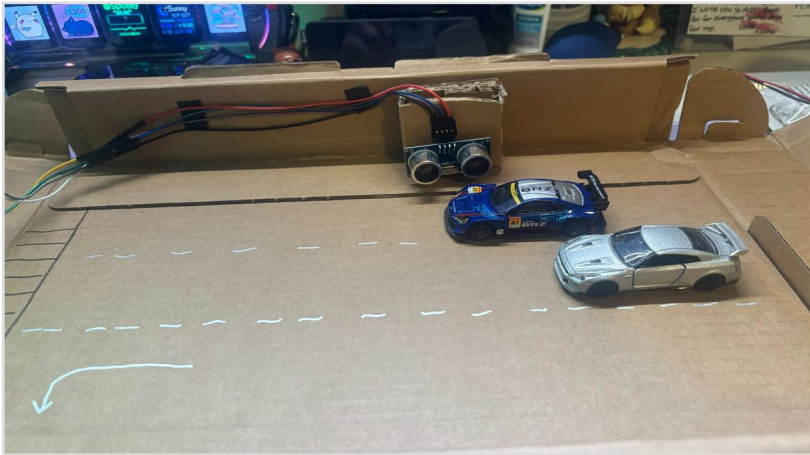
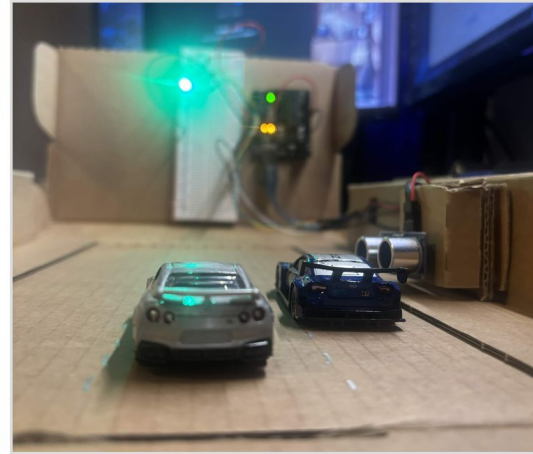
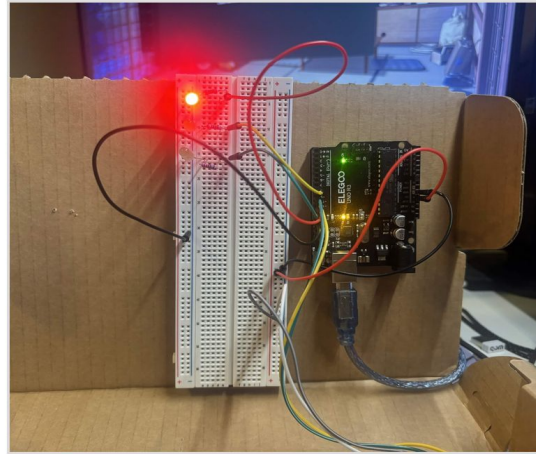
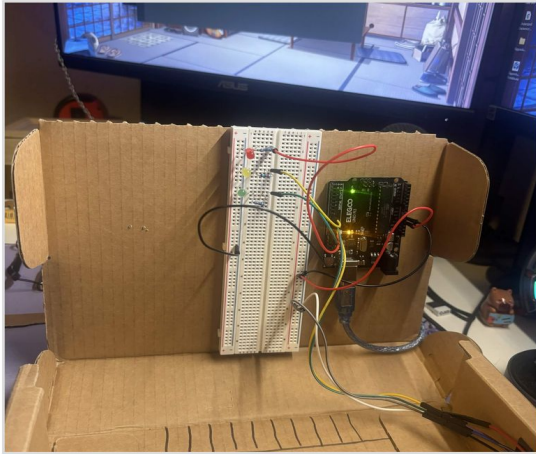
Resulting design decision

We chose a small breadboard model with Hot Wheels cars and an ultrasonic sensor. This allowed us to prove the traffic logic before attempting a larger intersection model.

MakerSpace impact

The project stayed within a realistic timeframe while still showing hardware, software, testing, and teamwork skills.

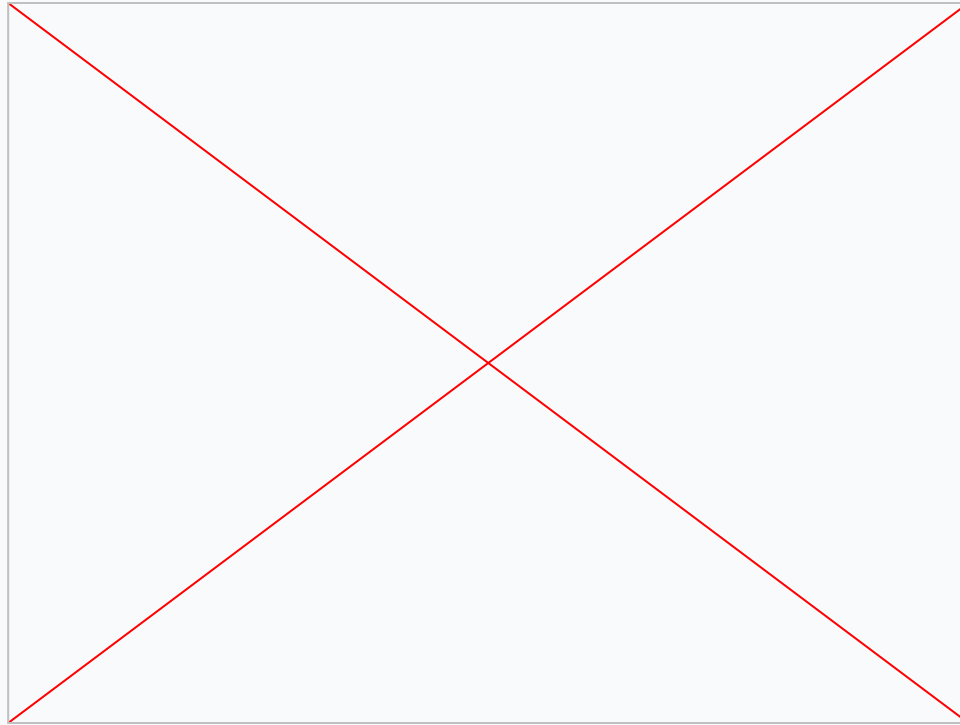
Demonstration setup



Demo steps

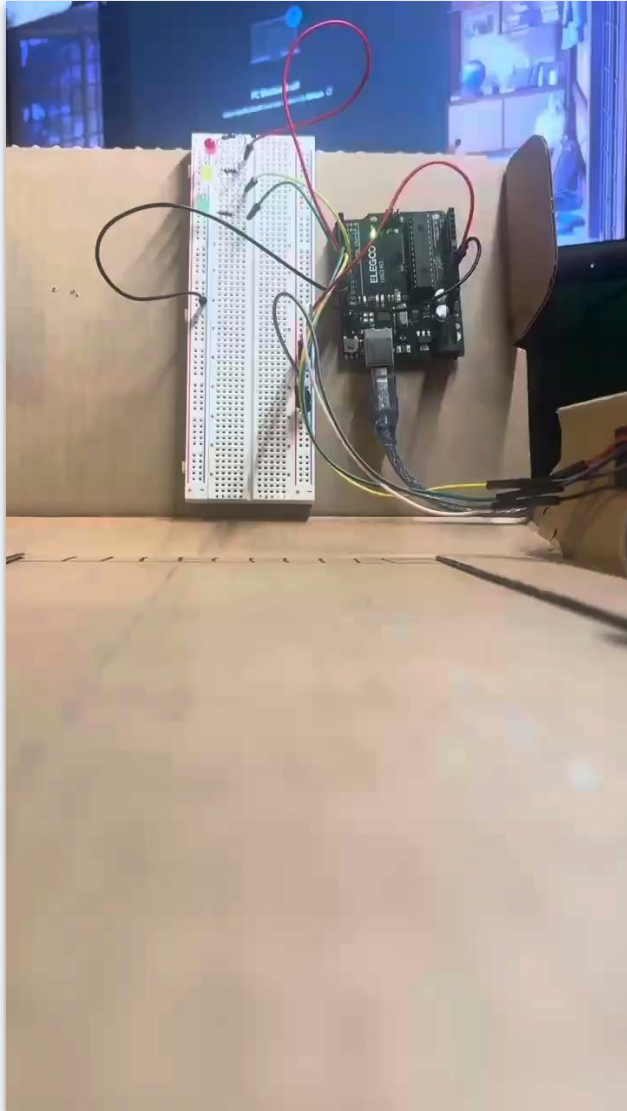
- 1 Run a 10-second count
- 2 Pass 0, 1–2, or 3+ cars through sensor zone
- 3 Arduino counts cars and selects timing
- 4 LED sequence changes timing

Demonstration setup (Continued)



Demo Video 1: No cars detected

Shortest timing because there is no simulated traffic demand.



0 cars → 3 s / 1 s / 3 s

```
-----  
Cars counted: 0  
Green time: 3 seconds  
Yellow time: 1 seconds  
Red time: 3 seconds  
-----  
GREEN light  
YELLOW light  
RED light  
Cycle complete. Restarting car count...  
  
Counting cars for 10 seconds...  
Distance: 16.77 cm  
Distance: 16.91 cm  
Distance: 16.91 cm  
Distance: 16.91 cm  
Distance: 16.93 cm  
Distance: 16.91 cm  
Distance: 16.91 cm
```

Demo Video 2: 1-2 cars detected

Medium timing because light traffic is detected.



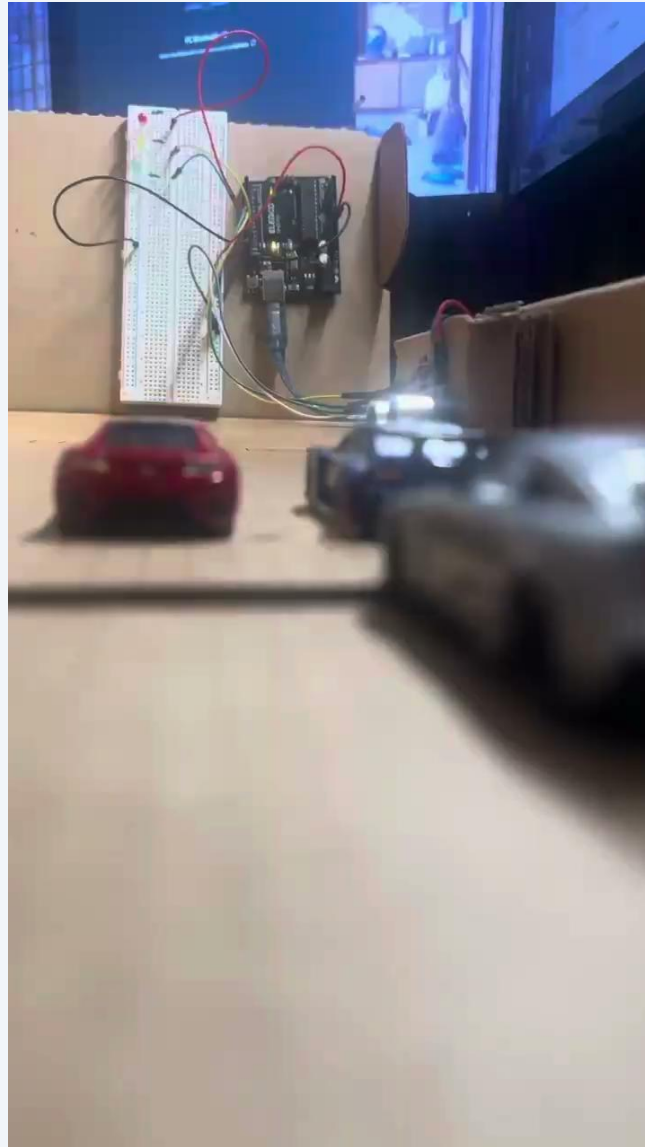
1-2 cars → 5 s / 3 s / 5 s

```
-----  
Cars counted: 1  
Green time: 5 seconds  
Yellow time: 3 seconds  
Red time: 5 seconds  
-----  
GREEN light  
YELLOW light  
RED light  
Cycle complete. Restarting car count...
```

```
Counting cars for 10 seconds...  
Distance: 17.41 cm  
Distance: 17.56 cm  
Distance: 17.56 cm  
Distance: 17.54 cm  
Distance: 17.54 cm  
Distance: 17.54 cm  
Distance: 17.56 cm
```

Demo Video 3: 3+ cars detected

Longest timing because the system detects the highest traffic level.



3+ cars → 8 s / 5 s / 8 s

```
-----  
Cars counted: 3  
Green time: 8 seconds  
Yellow time: 5 seconds  
Red time: 8 seconds  
-----  
GREEN light  
YELLOW light  
RED light  
Cycle complete. Restarting car count...
```

```
Counting cars for 10 seconds...  
Distance: 18.71 cm  
Distance: 18.85 cm  
Distance: 18.83 cm  
Distance: 18.83 cm  
Distance: 18.83 cm  
Distance: 18.83 cm  
Distance: 18.85 cm  
Distance: 18.85 cm  
Distance: 18.85 cm  
Distance: 18.85 cm  
Distance: 18.83 cm  
Distance: 18.85 cm
```

Final results: adaptive timing worked

The sensor-based prototype demonstrated three traffic response levels.

0 cars

No traffic

3 sec green · 1 sec yellow · 3 sec red

1–2 cars

Medium traffic

5 sec green · 3 sec yellow · 5 sec red

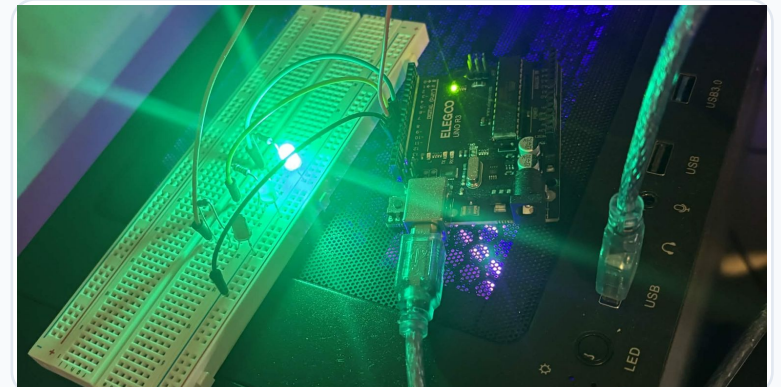
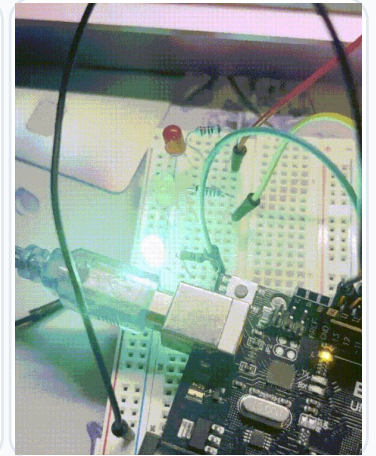
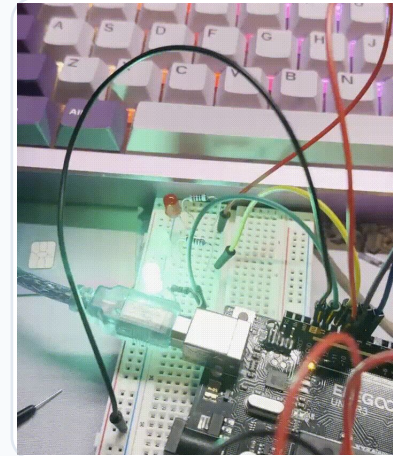
3+ cars

High traffic

8 sec green · 5 sec yellow · 8 sec red

What this proves

The prototype can detect simulated traffic demand and choose different signal timings. This demonstrates the core goal of a responsive traffic light system on a small scale.



Issues, fixes, and what we learned

The biggest challenge was reliable hardware-software communication.

```
clc;
clear;

a = arduino('COM3','Uno');

redLED = 'D10';
yellowLED = 'D9';
greenLED = 'D8';

while true
    % GREEN
    writeDigitalPin(a, greenLED, 1);
    writeDigitalPin(a, yellowLED, 0);
    writeDigitalPin(a, redLED, 0);
    pause(5);

    % YELLOW
    writeDigitalPin(a, greenLED, 0);
    writeDigitalPin(a, yellowLED, 1);
    writeDigitalPin(a, redLED, 0);
    pause(2);

    % RED
    writeDigitalPin(a, greenLED, 0);
    writeDigitalPin(a, yellowLED, 0);
    writeDigitalPin(a, redLED, 1);
    pause(5);
end
```

MATLAB setup issue

At first, MATLAB could not connect because the Arduino was not fully configured. Installing Matlab connector and arduinosetup was required.

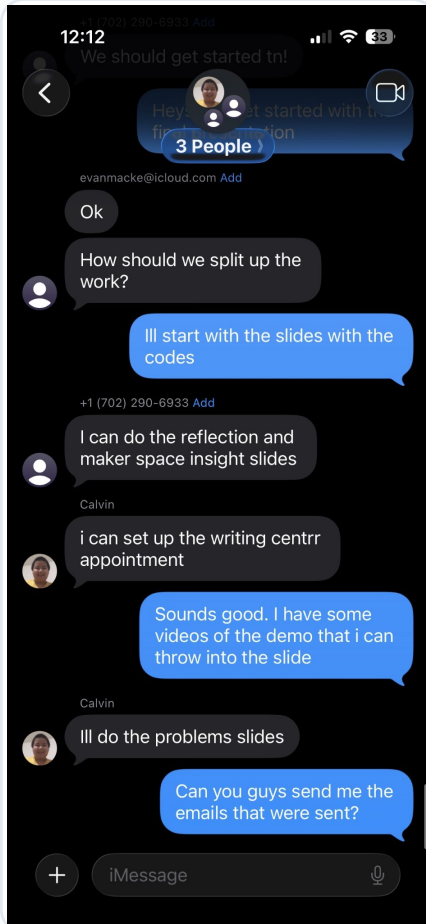
Counting delay

MATLAB had a harder time counting fast car passes because it communicates through the computer.

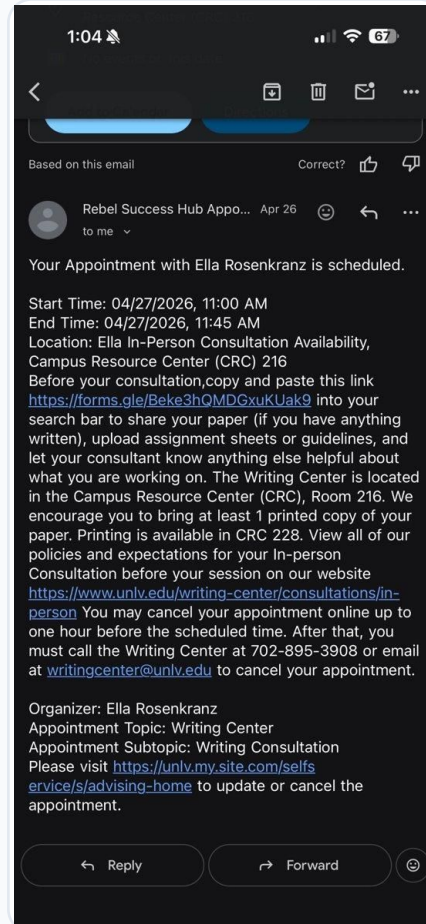
Final decision

Arduino IDE was used for the best final test results because the program runs directly on the board.

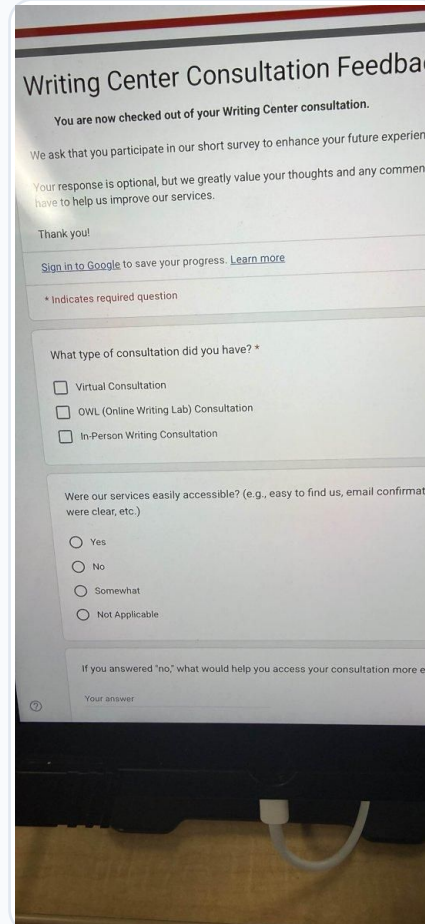
Teamwork and soft skills used throughout the project



Communication



Planning



Feedback



Documentation

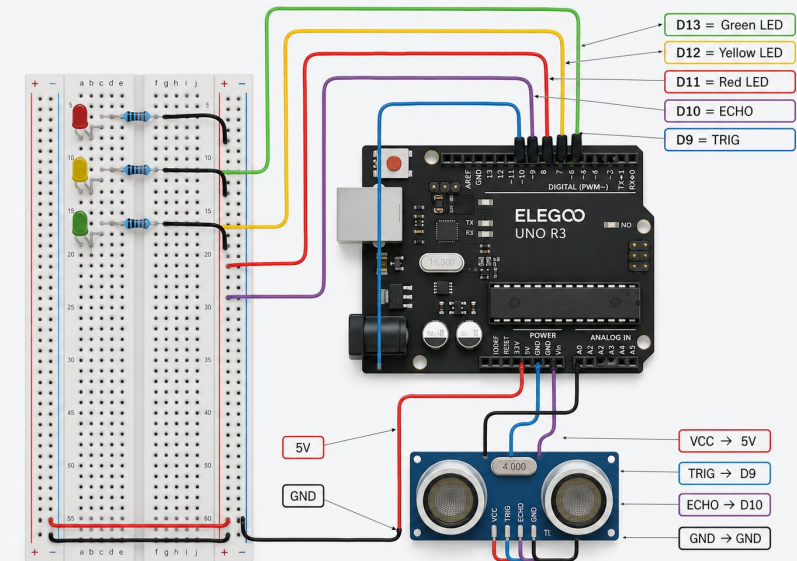
- For the most part, teamwork and communication were good.
- We had to work around everyone's busy schedules, but we made it work.
- We stayed organized by communicating, dividing tasks, and helping each other.
- We turned in all of our work on time and were able to complete the project together.

Conclusion

We built a working traffic light optimization simulator that responds to simulated traffic levels using an ultrasonic sensor and Arduino logic.

Career awareness and lessons learned

- Break large engineering ideas into testable prototypes
- Use feedback to narrow scope
- Compare tools and choose the most reliable workflow
- Document problems, changes, and final results clearly



Thank You

Questions?

Traffic Light Optimization Simulator

EGG 104-1003 · University of Nevada, Las Vegas

Zihan Chen · Calvin Le Zheng · Evan Macke · Nathaniel Lubas

